What we learned: Physics Callbacks and Singleton Pattern in a damage chain

Group: Awake()

Team members: Hang Ge, Qiyin Huang, Yanshuo Liu, Yixuan Liu, Xiangtian Ren, Sihan Wang (alphabetical by last name)

Scripts studied: Damage.cs, Health.cs, Enemy.cs, GameManager.cs, UIManager.cs

We chose to discuss two tightly coupled concepts: Physics Callbacks and the Singleton Pattern. Looking at either one alone does not explain the full damage chain. The physics callback starts the chain, and the Singleton is what lets the chain reach across unrelated scripts.

Concept 1: Physics Callbacks

When we first read Damage.cs, we looked for where DealDamage() gets called from. Tracing it back led us to three physics callback functions, none of which are called by our own code. Unity fires them automatically depending on how two objects make contact. All three are present in Damage.cs and each serves a different purpose:

- **OnTriggerEnter2D** fires **once** the moment a collider enters a trigger zone. Used for instant damage on first contact. In our project this is the callback used when a projectile hits an enemy (or vice versa).
- **OnTriggerStay2D** fires once per physics step (effectively every `FixedUpdate` for 2D physics) while a collider remains inside a trigger zone. Used for damage that should keep applying as long as two objects stay in contact, rather than only on the first frame.
- **OnCollisionEnter2D** fires **once** when two **non-trigger** colliders make contact. Used when damage should also apply between regular (non-trigger) colliders, not just trigger overlaps.

The key distinction is trigger vs. non-trigger: trigger colliders detect overlap without blocking movement; non-trigger colliders register contact through OnCollisionEnter2D instead. Having all three callbacks in the same Damage script lets a single component support several contact-based damage modes through Inspector flags, without needing separate scripts for each case.

Code evidence

```
// Damage.cs — fires once when a collider enters the trigger zone
private void OnTriggerEnter2D(Collider2D collision)
{
    if (dealDamageOnTriggerEnter)
    {
        DealDamage(collision.gameObject);
    }
}

// Damage.cs — fires once per physics step while a collider stays inside the
trigger
private void OnTriggerStay2D(Collider2D collision)
{
    if (dealDamageOnTriggerStay)
    {
        DealDamage(collision.gameObject);
    }
}

// Damage.cs — fires once when two non-trigger colliders make physical contact
private void OnCollisionEnter2D(Collision2D collision)
{
    if (dealDamageOnCollision)
    {
        DealDamage(collision.gameObject);
    }
}
```

Concept 2: Singleton Pattern

We noticed that the event chain reaches GameManager and UIManager through GameManager.instance and UIManager.instance. This is a first-wins Singleton: the first component to run Awake() claims the static instance, and any duplicate that runs Awake() later destroys itself. Any other script can then reach that single instance directly through GameManager.instance or UIManager.instance, without needing a reference assigned in the Inspector. GameManager uses `DestroyImmediate(this)` while UIManager uses `Destroy(this)`. In both cases only the script component is destroyed, not the GameObject it is attached to, so the GameObject keeps existing in the scene.

Code evidence

```
// GameManager.cs — Singleton setup in Awake()
public static GameManager instance = null;

private void Awake()
{
    if (instance == null)
    {
        instance = this;
    }
    else
    {
        DestroyImmediate(this);
    }
}

// Enemy.cs — every call site null-checks the instance before using it
private void AddToScore()
{
    if (GameManager.instance != null && !GameManager.instance.gameIsOver)
    {
        // AddScore is a static wrapper in GameManager that internally
        // forwards to the instance through the static `score` property.
        GameManager.AddScore(scoreValue);
    }
}
```

Event chain

```
Unity physics event: two colliders overlap
→ Damage.OnTriggerEnter2D(collision)        [Damage.cs]
→ DealDamage(collision.gameObject)          [Damage.cs]
→ Health.TakeDamage(damageAmount)           [Health.cs]
→ CheckDeath()                              [Health.cs]
→ Die()                                     [Health.cs]

If the damaged object is an enemy (Health.cs checks via
gameObject.GetComponent<Enemy>()):
→ Enemy.DoBeforeDestroy()                   [Enemy.cs]
→ Enemy.AddToScore()                        [Enemy.cs]
→ GameManager.AddScore(scoreValue)          [GameManager.cs]
→ GameManager.UpdateUIElements()            [GameManager.cs]
→ UIManager.instance.UpdateUI()             [UIManager.cs]
→ score changes on screen

If the damaged object is the player:
→ GameManager.instance.GameOver()           [GameManager.cs]
→ UIManager.instance.GoToPage(gameOverPageIndex) [UIManager.cs]
→ game over screen appears
```

Why this matters

Before reading these scripts, we assumed damage handling would be centralized, with one script checking every frame whether something had been hit. Instead, we found that a single physics event silently passes responsibility across five scripts without any of them needing to know the full picture. Damage.cs only knows it hit something. Health.cs only knows it lost health. Enemy.cs only knows it was destroyed. GameManager only knows a score changed. UIManager only knows the UI needs refreshing. Each script reacts to what it receives and passes the result on.

The Singleton pattern is what makes this handoff possible across unrelated scripts. GameManager.instance and UIManager.instance give any script a direct line to those managers without any setup in the Inspector. The risk is that this convenience depends entirely on the instance being there. If the manager object is missing or destroyed, calls through GameManager.instance or UIManager.instance may fail. That is why the project often checks whether the instance is null before using it.

We also noticed a precise distinction in Damage.cs: collision.gameObject is the object that was hit, while gameObject is the object owning the Damage script. Swapping these two would send damage

to the wrong target with no error message, making it a hard bug to find without knowing how Unity separates the two.

Improvement idea: Event-driven enemy spawning

Replace EnemySpawner's per-frame timer polling with Unity's scheduled invocation. EnemySpawner currently runs `CheckSpawnTimer()` inside `Update()` every frame just to see if enough time has passed since the last spawn. At a typical 60 FPS with a 2.5-second spawn delay, that is roughly 150 frame checks between spawns and only one of them actually does anything. `InvokeRepeating` lets Unity schedule the spawn callback directly on a fixed interval, which is closer to how the engine handles its own event timing.

```
// EnemySpawner.cs
private void OnEnable()
{
    InvokeRepeating(nameof(TrySpawn), spawnDelay, spawnDelay);
}

private void OnDisable()
{
    CancelInvoke(nameof(TrySpawn));
}

private void TrySpawn()
{
    if (currentlySpawned < maxSpawn || spawnInfinite)
    {
        SpawnEnemy(GetSpawnLocation());
    }
}
// The timing logic in Update() and CheckSpawnTimer() can be replaced.
```

This removes one `Update()` call per spawner and hands the timing back to Unity, which is the engine-driven approach the rest of this post is about.

Improvement idea: Decoupling score updates with UnityEvent

Replace GameManager's direct UI call with a UnityEvent so the score system stops knowing about UIManager. Right now `GameManager.AddScore()` calls `UpdateUIElements()`, which calls

`UIManager.instance.UpdateUI()`. That means adding a new score-driven UI element (a high-score popup, a streak counter) would require a code change inside GameManager. A UnityEvent lets any listener subscribe in the Inspector without GameManager needing to know about it.

```
// GameManager.cs
using UnityEngine.Events;

[Header("Score Events")]
public UnityEvent<int> onScoreChanged = new UnityEvent<int>();

public static void AddScore(int scoreAmount)
{
    score += scoreAmount;
    if (score > instance.highScore)
    {
        SaveHighScore();
    }
    instance.onScoreChanged.Invoke(score);   // replaces UpdateUIElements()
}
```

UIManager (or any other listener) can then subscribe to `onScoreChanged` in the Inspector and run its own update logic when the score changes. New listeners can be added later without touching GameManager. Note that this only removes the direct dependency between GameManager and UIManager. GameManager still goes through `instance` to invoke the event, so the Singleton coupling identified in Concept 2 is unchanged. The point of this change is to stop GameManager from knowing about specific UI scripts, not to remove the
Singleton itself.

Sources

- Unity Scripting API: MonoBehaviour: background for both concepts; every script we studied inherits from MonoBehaviour.
- Unity Manual: Event function execution order: used to confirm that `Awake()` runs before `Start()`, which is why GameManager.cs and UIManager.cs set up the Singleton inside `Awake()` (Concept 2).
- Unity Scripting API: OnTriggerEnter2D: supports Concept 1 and the start of the event chain.
- Unity Scripting API: OnTriggerStay2D: supports the per-physics-step behavior described in Concept 1.

- Unity Scripting API: OnCollisionEnter2D: supports the trigger vs. non-trigger distinction in Concept 1.
- Unity Scripting API: MonoBehaviour.InvokeRepeating: used in the event-driven enemy spawning improvement to replace EnemySpawner's `Update()` polling.
- Unity Scripting API: UnityEvent: used in the score decoupling improvement to broadcast score changes.

Reflection

Reading these scripts changed how we think about finding bugs. If damage is not being applied, there is no point looking at TakeDamage() first. The real question is whether OnTriggerEnter2D() fired at all, which depends on collider settings, trigger flags, and team IDs. The event chain gives us a map: if the chain breaks at any step, we know exactly where to look.

The part that surprised us most was how easy it is to mistake helper functions for Unity events. DealDamage(), TakeDamage(), and DoBeforeDestroy() handle most of the visible work, so they look like the important entry points, but Unity never touches any of them. The only function Unity actually calls in this chain is OnTriggerEnter2D(). Everything else is just one function calling the next.